Generative AI Fundamentals

Core concepts explained with practical examples

Beginner-friendly study guide • Concepts, examples, and practical exercises

1. AI, machine learning, deep learning and GenAI

Artificial intelligence (AI) is the broad field of building systems that perform tasks associated with intelligence. Machine learning (ML) learns patterns from data. Deep learning uses neural networks with multiple layers. Generative AI creates new content such as text, code, images or audio.

Example: a spam classifier predicts a label; a generative model drafts an email. Some systems combine both.

2. Neural networks and transformers

A neural network contains layers of mathematical operations whose parameters are adjusted during training. A transformer is a neural-network architecture that uses attention to model relationships among tokens.

Attention helps the model weigh which parts of the input are relevant to each other. Parameters are learned numerical values. Training adjusts parameters using data; inference is using the trained model to produce outputs.

3. Tokens, tokenization and context windows

A tokenizer converts text into token IDs. A token may be a word, part of a word, punctuation or other symbol. Token counts are not the same as word counts.

The context window is the amount of input and output the model can handle in one request. Very long prompts can be expensive or exceed the model limit. RAG helps by retrieving only relevant document passages instead of sending every document.

4. How an LLM generates text

An autoregressive language model predicts a probability distribution for the next token given previous tokens. A decoding strategy selects a token, adds it to the sequence and repeats until a stop condition.

Temperature influences randomness. Max output tokens limits response length. Stop conditions end generation. These settings change behavior but do not guarantee correctness.

5. Embeddings and semantic similarity

An embedding is a list of numbers representing a piece of text in a vector space. Texts with similar meanings often have vectors that are close under a chosen similarity or distance measure.

Embeddings are useful for semantic search, clustering and recommendations. They do not contain a human-readable explanation of a sentence, and similarity is not proof that a result is true.

```
Question: 'How do I reset my password?'\nRelevant document: 'Steps for changing a forgotten login credential.'\
```

6. Prompt engineering

A prompt is the input and instructions supplied to a model. Clear prompts specify the task, context, constraints and desired output format.

```
Role: You are a helpful study assistant.\nTask: Explain the concept in simple language.\nContext: [trusted note
```

Good prompting improves consistency, but does not eliminate hallucinations. Validate outputs, especially when decisions have real consequences.

7. Hallucinations, grounding and evaluation

A hallucination is an answer that sounds plausible but is unsupported or incorrect. Grounding means tying an answer to reliable evidence, such as retrieved documents or verified data.

Evaluate with representative questions: correctness, relevance, completeness, citation accuracy, latency, cost and refusal behavior. Keep test cases and compare versions instead of relying only on a few impressive demos.

8. Fine-tuning vs RAG vs prompting

Prompting: changes instructions at request time; fast to try, but does not permanently teach the model new facts.

RAG: retrieves relevant external information at question time; useful for changing or private knowledge and source citations.

Fine-tuning: adjusts model parameters using examples; useful for behavior, style or task patterns when justified by data and evaluation. It is not usually the simplest way to keep factual documents current.

9. Local models vs hosted APIs

A local model runs on your computer or infrastructure. Benefits can include offline operation and greater control, but hardware, speed and model quality are constraints. A hosted API avoids local model serving but requires network access and may incur usage charges.

Your Qwen2.5-0.5B-Instruct project is a local-model example. A small model can be useful for learning, but its answers may be weaker than larger models. Check licensing and data handling before using any model commercially.

10. GenAI system building blocks

• Model: generates or transforms content.

• Tokenizer: maps text to token IDs and back.

• Prompt template: creates consistent instructions.

• Retriever: finds relevant evidence.

• Embedding model: represents text as vectors.

• Vector index/database: stores vectors and enables similarity search.

• Orchestration: connects steps, tools and control flow.

• UI/API: lets users or other programs interact with the system.

• Evaluation and monitoring: checks quality, failures, latency and cost.

These parts are optional building blocks, not a requirement to use every framework in every project.

11. Responsible and secure GenAI

Treat user inputs and retrieved documents as untrusted. Avoid exposing secrets in prompts or logs. Apply access control before retrieval, validate tool arguments, limit permissions, and test prompt injection and data leakage.

Do not send confidential documents to a hosted provider unless your organization has approved the data flow. Review provider retention, region, access and contractual requirements.

12. Exercises

• Explain tokens, context windows and temperature in your own words.

• Compare prompting, RAG and fine-tuning for a company FAQ.

• Build a prompt that answers only from supplied context.

• Create 20 test questions and score answers for correctness and grounding.

• Compare a local model and a hosted API on the same test set.

Official documentation and further reading

Hugging Face LLM Course — free learning materials about LLMs and the Transformers ecosystem
<link href='https://huggingface.co/learn' color='blue'>https://huggingface.co/learn</link>

Hugging Face course repository — open-source course materials
<link href='https://github.com/huggingface/course' color='blue'>https://github.com/huggingface/course</link>

OpenAI embeddings guide — embeddings and semantic similarity
<link href='https://developers.openai.com/api/docs/guides/embeddings' color='blue'>https://developers.openai.com/api/docs/guides/embeddings</link>

OpenAI Retrieval guide — vector stores and semantic search
<link href='https://developers.openai.com/api/docs/guides/retrieval' color='blue'>https://developers.openai.com/api/docs/guides/retrieval</link>